

AI-Enabled Quality Gates in CI/CD

Rivaldo Pinto^{1*}† and Rodrigo Santos^{1†}

^{1*}Department of Computer Science, University of Beira Interior, Rua
Marquês D'Ávila e Bolama, 6201-001, Portugal.

*Corresponding author(s). E-mail(s): rivaldo.pinto@ubi.pt;

Contributing authors: rodrigo.fl.santos@ubi.pt;

†These authors contributed equally to this work.

Abstract

Os pipelines de *Continuous Integration* e *Continuous Delivery* (CI/CD) incorporam cada vez mais *gates* de qualidade baseados em inteligência artificial para automatizar decisões relacionadas com testes, qualidade de código e avaliação de risco. Apesar de estas ferramentas prometirem ciclos de *feedback* mais rápidos e maior qualidade, introduzem também novos riscos, nomeadamente falsos positivos, falsos negativos e uma dependência excessiva da automação. Este trabalho apresenta um estudo comparativo entre *gates* de qualidade manuais e assistidos por IA, desenvolvido sobre um pipeline real com GitHub Actions e SonarCloud. Foram executados oito cenários estruturados de *Pull Request*, cobrindo situações de código correcto, duplicação, cobertura baixa, defeitos subtis, falso positivo com *override* e falso negativo. Os resultados mostram que os dois *gates* concordaram em 50% dos casos, o *gate* com IA produziu uma taxa de aprovação de 37,5% contra 87,5% do *gate* manual, e foi registado um *override* motivado por um falso positivo. O caso mais crítico foi o do falso negativo, onde ambos os *gates* aprovaram código com um defeito funcional real. As conclusões reforçam que a automação com IA é um complemento valioso à supervisão humana, mas não a substitui.

Keywords: Software Quality Assurance, Automation, Human-AI Collaboration, CI/CD Pipelines, Automation Bias, Responsible Automation.

1 Enquadramento

O presente trabalho encontra-se incluído na Unidade Curricular de Qualidade de Software, pertencente ao Segundo Ciclo do Curso de Engenharia Informática, na

Universidade da Beira Interior. O seu propósito é discutir e comparar os métodos automáticos para validação de código em *pipelines* de CI/CD, com destaque na integração de sistemas de qualidade que possuem suporte da inteligência artificial, através da análise do impacto que diferentes tipos de *quality gate* possuem no processo de integração contínua e também suas limitações e riscos.

2 Introdução

O desenvolvimento de *software* moderno depende cada vez mais de *pipelines* de *Continuous Integration* e *Continuous Delivery* (CI/CD) para automatizar os processos de construção, teste e entrega de aplicações. Neste contexto, os *quality gates* assumem um papel fundamental, funcionando como mecanismos de controlo que determinam se uma alteração de código reúne as condições necessárias para ser integrada no ramo principal de desenvolvimento. Tradicionalmente, estes mecanismos baseiam-se na execução de testes unitários, métricas de cobertura de código e revisão manual realizada por programadores experientes. Contudo, como o SonarCloud, introduziu uma nova abordagem à validação da qualidade do código. Nestes ambientes, a análise automatizada passa a complementar o processo de tomada de decisão, identificando potenciais defeitos, vulnerabilidades, problemas de manutenibilidade e outros padrões considerados prejudiciais à qualidade do *software*.

Apesar dos benefícios associados a esta automação, a utilização de *quality gates* assistidos por IA levanta desafios importantes. Os *gates* com IA podem bloquear código funcionalmente correcto com base em critérios estruturais (falso positivo), ou aprovar código com defeitos reais de lógica de negócio que escapam à análise estática (falso negativo). Entender quando e por que esses erros ocorrem é fundamental para criar *pipelines* CI/CD confiáveis e ajustar a confiança dos programadores nas ferramentas de automação.

Este trabalho procura responder às seguintes questões de investigação:

RQ1 *De que forma os gates de qualidade com IA afectam os resultados do pipeline?*

RQ2 *Qual é a frequência de overrides e de decisões falsas?*

As principais contribuições deste trabalho são a implementação de um *pipeline* CI/CD experimental e reproduzível, a recolha de dados empíricos obtidos através da execução de oito cenários distintos de *Pull Request*, e a análise dos compromissos existentes entre automação, qualidade do código e confiança nas decisões assistidas por IA.

3 Conceitos do Projeto

O desenvolvimento moderno de *software* depende fortemente de práticas que garantem rapidez, fiabilidade e ciclos de feedback curtos, e, neste contexto, os *pipelines* de CI/CD assumem um papel central, permitindo que cada alteração seja construída, testada e validada de forma consistente. Esta secção apresenta os conceitos fundamentais necessários para compreender o estudo realizado, começando pela Integração Contínua e Entrega Contínua, que constituem a base operacional do *pipeline* analisado.

3.1 Pipelines CI/CD

A Integração Contínua (CI) é uma prática em que os programadores integram alterações de código num repositório partilhado várias vezes ao dia. Cada integração desencadeia automaticamente um conjunto de tarefas, nomeadamente compilação, execução de testes e análise de qualidade, garantindo que o sistema permanece funcional e que erros são detetados o mais cedo possível [1]. O objetivo principal da CI é reduzir o risco de integração, evitar acumulação de defeitos e fornecer feedback rápido sobre o impacto de cada alteração.

A Entrega Contínua (CD) estende este processo ao automatizar a preparação do software para ser disponibilizado em produção [2]. Embora o *deploy* final possa exigir aprovação humana, todo o *pipeline* até esse ponto é automatizado, desde construção, testes e empacotamento, a validações de qualidade. Isto permite que o software esteja sempre num estado “pronto a lançar”, reduzindo o tempo entre desenvolvimento e disponibilização.

Em conjunto, CI e CD criam um fluxo de desenvolvimento automatizado, repetível e fiável, reduzindo o tempo de *feedback*, aumentando a confiança nas alterações e permitindo que equipas entreguem *software* de forma mais rápida e segura. No contexto deste projeto, o *pipeline* CI/CD é o mecanismo que executa tanto o *gate* manual (baseado em testes) como o *gate* com IA (baseado em análise estática com *SonarCloud*), permitindo comparar diretamente o comportamento de ambos.

3.2 Quality Gates

Um *gate* de qualidade é um ponto de controlo formal dentro de um pipeline CI/CD que impõe um conjunto mínimo de critérios que o código tem de cumprir antes de poder avançar. Os critérios podem incluir a taxa de aprovação dos testes, a cobertura de código, a presença de bugs ou vulnerabilidades, e os resultados de ferramentas de análise estática, com o objetivo de garantir que cada contribuição mantém ou melhora o nível de qualidade do projeto. Neste estudo distinguem-se dois tipos:

- **Gate manual:** baseia-se na execução de testes unitários com *pytest* e na análise dos relatórios de cobertura. Este gate avalia a correção funcional do código, respondendo à pergunta: “O código funciona como esperado?”. Quando os testes estão bem definidos, este mecanismo é eficaz na deteção de defeitos lógicos e regressões.
- **Gate com IA:** Baseado na análise estática realizada pelo *SonarCloud*, este gate avalia a qualidade estrutural do código, incluindo duplicação, complexidade, manutenibilidade e cobertura do código novo. Responde à pergunta: “O código está bem estruturado e é sustentável a longo prazo?”. Embora seja eficaz na deteção de problemas arquiteturais, não avalia a lógica de negócio.

Assim, a combinação destes dois *gates* permite uma avaliação mais completa, integrando tanto a perspetiva funcional como a estrutural da qualidade do *software*.

3.3 SonarCloud

O SonarCloud [3] é uma plataforma de análise estática que aplica regras e heurísticas para classificar problemas por tipo (*bugs*, vulnerabilidades, *code smells*) e severidade

(bloqueador, crítico, major, minor), avaliando automaticamente a qualidade estrutural do código. O seu mecanismo de *quality gate* define os critérios mínimos para aprovação de um *Pull Request*, que podem incluir limites de duplicação, ausência de *bugs* críticos e níveis mínimos de cobertura de testes no código novo. No contexto deste projeto, o *quality gate* foi configurado para exigir uma classificação de Manutenibilidade A e uma cobertura mínima de 80% no código, garantindo que apenas alterações bem estruturadas e adequadamente testadas são aprovadas.

4 Estado da Arte e Trabalhos Relacionados

Ao longo dos últimos anos, a investigação sobre CI/CD e mecanismos de validação automática tem vindo a crescer significativamente, de modo a acompanhar a adoção generalizada de pipelines automatizados em projetos de *software*. Como tal, existem hoje diversos estudos que analisam o impacto da automação na qualidade do código, na produtividade das equipas e na fiabilidade das integrações.

Zampetti et al. [4] analisaram falhas de construção em projetos *open-source* e concluíram que uma parte substancial dessas falhas está associada a violações detetadas por ferramentas de análise estática. Este resultado reforça a importância de *quality gates* automáticos, especialmente em ambientes onde múltiplos programadores contribuem simultaneamente para o mesmo repositório.

Hilton et al. [5] estudaram a adoção de CI em larga escala e demonstraram que equipas que utilizam pipelines automatizados tendem a lançar *software* com maior frequência e com menos defeitos. O estudo destaca que a automação reduz o tempo de *feedback* e aumenta a confiança no processo de integração, dois aspetos diretamente relacionados com o *pipeline* utilizado neste trabalho.

Vassallo et al. [6] investigaram como os programadores reagem a falhas de CI e observaram que, sob pressão de tempo, é comum ignorar ou contornar gates que falham, especialmente quando os programadores acreditam que a falha não é relevante para a funcionalidade. Este comportamento está alinhado com o cenário de *override* documentado no PR11, onde o programador optou por integrar código funcional apesar da reprovação do *gate* com IA.

O nosso trabalho distingue-se por comparar explicitamente *gates* manuais e com IA dentro do mesmo *pipeline*, por identificar casos concretos de falso positivo e falso negativo, e por documentar a decisão de *override* e as suas motivações.

5 Metodologia

A metodologia adotada neste estudo foi construída para permitir uma comparação rigorosa entre *gates* de qualidade manuais e assistidos por IA num ambiente CI/CD realista. Para tal, foi desenvolvido um *pipeline* reproduzível, baseado em *GitHub Actions* e integrado com o *SonarCloud*, capaz de avaliar automaticamente cada *Pull Request* segundo critérios funcionais e estruturais. A abordagem inclui a definição da aplicação em teste, a configuração dos *workflows*, a criação de cenários experimentais controlados e a recolha sistemática de métricas relevantes. Esta secção descreve em detalhe cada um destes elementos, permitindo compreender como os dados foram obtidos e como as questões de investigação foram operacionalizadas.

5.1 Configuração do Pipeline

O projeto experimental foi implementado sobre um *pipeline* CI/CD real, alojado no GitHub, visando obter uma comparação direta da dinâmica de funcionamento de um *gate* manual e um *gate* com IA. Para a sua realização, foi desenvolvida uma aplicação de teste simples de sistema de gestão de inventário em *Python*, a qual foi integrada num repositório do GitHub, onde cada *Pull Request* inicia dois *workflows* distintos, ambos definidos com *GitHub Actions*.

O primeiro *workflow*, *ci-manual-only.yml*, consistia apenas em execução de testes unitários com *pytest*, cálculo de cobertura do código e, finalmente, aprovação/reprovação em função do resultado dos testes unitários. Este *workflow* representa o *gate* manual, centrado na verificação da validade funcional.

O segundo *workflow*, *ci-with-sonar.yml* representa o *gate* com IA. Além dos passos da *ci-manual-only.yml*, foi adicionada uma etapa de análise estática com o *SonarCloud*. Esta análise consiste em avaliação de duplicação, complexidade, manutenibilidade e cobertura do código novo no projeto. Estas informações são avaliadas conforme o *quality gate* configurado para o projeto.

Ambos os *workflows* foram iniciados paralelamente em cada *Pull Request*, o que proporciona condições controláveis para a análise das decisões tomadas por cada tipo de *gate* e ainda de divergências, falsos positivos, falsos negativos e *overrides*.

A Figura 1 apresenta a arquitetura geral do pipeline.

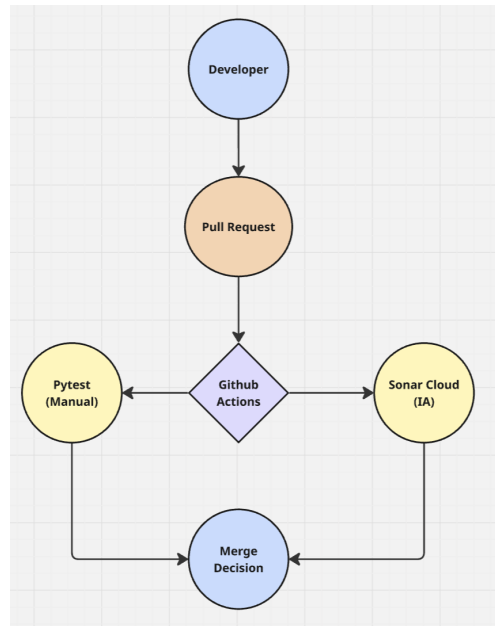


Fig. 1 *Pipeline* CI/CD utilizado no estudo experimental.

5.2 Aplicação em Teste

A aplicação utilizada neste trabalho consiste num sistema simplificado de gestão de inventário desenvolvido em Python. A funcionalidade principal encontra-se implementada na classe `Inventory`, que disponibiliza operações para registo de produtos (`add_product`), atualização de *stock* (`update_stock`), remoção de unidades (`remove_stock`), identificação de produtos com *stock* reduzido (`get_low_stock`) e cálculo do valor total do inventário (`get_total_value`).

A adoção de uma aplicação intencionalmente simples permitiu isolar a lógica de negócio, garantindo que as avaliações produzidas pelos *Quality Gates* refletiam exclusivamente as alterações e defeitos introduzidos em cada cenário experimental, sem interferência de componentes ou dependências externas complexas.

5.3 Processo Experimental

Para cada cenário foi criado um *branch* dedicado a partir do ramo principal, seguido de um *Pull Request* que desencadeou automaticamente os dois *workflows*. Os resultados do *pytest* e do SonarCloud (aprovação ou reprovação dos testes unitários, cobertura de código, decisão do SonarCloud) foram registados no ficheiro `data/metrics.json`. Os campos automáticos (`tests_passed`, `coverage`, `ai_decision`) foram preenchidos pelo *script* `save_metrics.py`, enquanto que os campos manuais (`manual_decision`, `override`, `override_reason`, `bugs_after_merge`) foram preenchidos pelo programador com base na análise de cada *Pull Request*.

5.4 Cenários Experimentais

Para avaliar o comportamento dos mecanismos de validação, foram definidos oito cenários independentes. Cada cenário foi implementado através de um *Pull Request* (PR) específico, com o objetivo de reproduzir situações reais e frequentes no desenvolvimento de *software*.

A Tabela 1 descreve o foco de cada cenário testado.

Table 1 Cenários experimentais e comportamento esperado dos *Quality Gates*.

PR	Cenário	Manual Gate	AI Gate
5	Baseline	PASS	PASS
6	Bug Funcional	FAIL	FAIL
7	Validação em Falta	PASS	PASS
8	Duplicação de Código	PASS	FAIL
9	Cobertura Baixa	PASS	FAIL
10	Defeito Subtil	PASS	FAIL
11	Override (Potencial Falso Positivo)	PASS	FAIL
15	Falso Negativo	PASS	PASS

Estes cenários foram criados para cobrir diferentes resultados possíveis num pipeline CI/CD, incluindo situações de concordância entre mecanismos de validação, reprovações automáticas, *overrides*, e ainda potenciais falsos positivos e falsos negativos.

5.5 Ordem Numérica dos Cenários

A numeração dos *Pull Requests* apresentada neste estudo inicia-se no PR5. Durante uma fase preliminar do projeto foi utilizada uma aplicação baseada numa calculadora simples para validar a configuração inicial do pipeline CI/CD e das ferramentas de automação. No entanto, verificou-se posteriormente que essa aplicação não permitia reproduzir de forma adequada todos os cenários necessários para responder às questões de investigação, nomeadamente situações de *override*, falsos positivos e falsos negativos. Por esse motivo, optou-se por substituir a aplicação inicial por um sistema de gestão de inventário, mais adequado aos objetivos experimentais do estudo.

Adicionalmente, o cenário de *Override* (PR11) foi integrado primeiro no ramo principal, introduzindo intencionalmente alguma dívida técnica. Devido a esta integração, as tentativas seguintes de implementar o cenário de falso negativo (do PR12 ao PR14) não funcionaram como esperado. O SonarCloud continuava a reprovar estes *Pull Requests*, não devido ao defeito lógico que se pretendia testar, mas sim devido ao histórico de *code smells* introduzido pelo PR11.

Para resolver este problema e garantir que a ferramenta analisava apenas o comportamento da nova função (`apply_discount`), foi necessário reverter o código principal para o estado limpo em que se encontrava antes da integração do PR11. Só depois desta reversão foi possível executar e validar o cenário de falso negativo com sucesso no PR15, preservando assim a validade de todo o teste.

5.6 Métricas Recolhidas

A recolha de métricas foi realizada de maneira a permitir a comparação entre o *gate* manual e o *gate* com IA, bem como a identificação de falsos positivos, falsos negativos e situações de *override*. Para cada *Pull Request*, o script `save_metrics.py` registou automaticamente os resultados produzidos pelos *workflows*, enquanto os campos que exigiam interpretação humana foram completados manualmente. Estas métricas encontram-se representadas na Tabela 2, e podem ser agrupadas nas três categorias descritas a seguir:

- Métricas funcionais, derivadas da execução dos testes unitários (por exemplo, `tests_passed` e `coverage`), que permitem avaliar a correção do comportamento do código.
- Métricas estruturais, provenientes da análise estática do SonarCloud (por exemplo, `ai_decision`), que avaliam a qualidade técnica e a manutenibilidade das alterações.
- Métricas humanas, preenchidas manualmente (`manual_decision`, `override`, `override_reason`, `bugs_after_merge`), que capturam o julgamento do programador e os efeitos reais das decisões de merge.

Table 2 Métricas recolhidas por *Pull Request*

Campo	Tipo	Fonte
<code>tests_passed</code>	Booleano	<i>pytest</i> (automático)
<code>coverage</code>	Decimal	<i>pytest-cov</i> (automático)
<code>ai_decision</code>	Texto	SonarCloud (automático)
<code>manual_decision</code>	Texto	Programador (manual)
<code>override</code>	Booleano	Programador (manual)
<code>override_reason</code>	Texto	Programador (manual)
<code>bugs_after_merge</code>	Inteiro	Programador (manual)

Os campos automáticos `tests_passed`, `coverage` e `ai_decision`, foram recolhidos diretamente dos *workflows*, garantindo consistência e eliminando qualquer *bias* humano. Já os campos manuais `manual_decision`, `override`, `override_reason` e `bugs_after_merge`, foram preenchidos após análise individual de cada *Pull Request*, permitindo documentar explicitamente divergências entre os *gates* e validar na prática a ocorrência de defeitos após o *merge*.

As métricas foram analisadas para responder às duas questões de investigação: a RQ1 utiliza as taxas de aprovação e reprovação e a taxa de concordância entre os dois *gates*, enquanto que a RQ2 utiliza a frequência de *overrides*, a precisão, o *recall* e o F1 do *gate* com IA, tomando o *gate* manual como referência.

6 Resultados

Esta secção apresenta os resultados cenário a cenário, seguidos de uma análise consolidada que responde às duas questões de investigação.

6.1 PR5 - Baseline

O primeiro cenário estabeleceu a referência positiva do estudo. O código introduzido estava correcto e os testes existentes cobriam todas as funcionalidades implementadas.

- **Gate manual:** todos os testes passaram com uma cobertura global de 74,19%.
- **Gate IA:** o SonarCloud aprovou o *Pull Request* sem identificar problemas no código novo.
- **Decisão:** aprovação e *merge* sem *override*.

Este resultado confirma que, em condições normais de desenvolvimento, os dois *gates* concordam e o pipeline funciona de forma fluida.

6.2 PR6 - Bug Funcional

Neste cenário foi introduzido um defeito lógico óbvio no método `update_stock`: a operação de adição foi substituída por uma atribuição a zero, fazendo com que o stock nunca fosse actualizado correctamente.

Listing 1 Bug introduzido no método `update_stock`

```
def update_stock(self, name, quantity):
```



```

    if name not in self.products:
        raise KeyError(self.ERR_NOT_FOUND)
    # bug: devia ser += quantity
    self.products[name]["quantity"] = 0

```

- **Gate manual:** o teste `test.update_stock` falhou: o valor esperado era 10 mas o resultado foi 0.
- **Gate IA:** o SonarCloud reprovou por cobertura insuficiente e problemas de manutenibilidade.
- **Decisão:** reprovação sem *merge*. Ambos os *gates* detectaram o problema.

Este cenário demonstra o funcionamento esperado do pipeline: quando o código tem um defeito funcional claro, ambos os *gates* bloqueiam a integração.

6.3 PR7 - Validação em Falta

Este cenário testou o comportamento dos *gates* quando são adicionados casos de validação em falta na lógica existente, sem introduzir defeitos funcionais directos.

- **Gate manual:** todos os testes passaram com 75,86% de cobertura.
- **Gate IA:** o SonarCloud aprovou o *Pull Request*.
- **Decisão:** aprovação e *merge* sem *override*.

A concordância entre os dois *gates* neste cenário mostra que, para alterações de validação bem testadas, o pipeline opera sem fricção.

6.4 PR8 - Duplicação de Código e Dívida Técnica

Neste cenário foi introduzida dívida técnica intencional: foram adicionadas funções com forte duplicação lógica, violando o princípio DRY (*Don't Repeat Yourself*), sem os correspondentes testes unitários.

- **Gate manual:** os testes existentes passaram na totalidade. A cobertura global ficou em 53,19%, mas os testes de regressão não detectaram a degradação arquitectural.
- **Gate IA:** o SonarCloud reprovou com cobertura de código novo de apenas 12,5% (inferior ao limiar de 80%) e com Classificação de Manutenibilidade C, por duplicação de literais e uso de excepções genéricas.
- **Decisão:** reprovação. O *merge* não foi efectuado.

Este resultado ilustra uma das principais mais-valias do *gate* com IA: detectar degradação arquitectural que os testes de regressão tradicionais não conseguem identificar.

6.5 PR9 - Cobertura Baixa

Neste cenário foi adicionado código novo sem os testes correspondentes, fazendo com que a cobertura do código novo ficasse muito abaixo do limiar configurado no SonarCloud.

- **Gate manual:** os seis testes existentes passaram, com cobertura global de 60,0%.

- **Gate IA:** o SonarCloud reprovou, pois a cobertura do código novo foi de apenas 11,11%, muito abaixo do limiar de 80%.
- **Decisão:** reprovação. O *merge* não foi efectuado.

Este cenário mostra que o *gate* com IA detecta não só a qualidade do código existente, mas também a ausência de testes para funcionalidades novas, algo que o *gate* manual não consegue fazer sem testes explícitos.

6.6 PR10 - Defeito Subtil de Lógica de Negócio

Para testar os limites da análise estática, foi introduzido um erro lógico numa funcionalidade de cálculo de imposto: em vez de somar o imposto ao preço base, o código subtraía-o. Foi também criado um teste que validava propositadamente o resultado incorrecto.

- **Gate manual:** todos os testes passaram, incluindo o teste que validava o comportamento defeituoso. Cobertura de 76,32%.
- **Gate IA:** o SonarCloud reprovou, mas não por detectar o erro lógico. A reprovação deveu-se a problemas de estilo e design que reduziram a Classificação de Manutenibilidade abaixo do limiar exigido.
- **Decisão:** reprovação por razões estruturais. O defeito funcional não foi detectado por nenhum dos *gates*.

Este é um resultado especialmente revelador: o *gate* com IA bloqueou o código, mas pela razão errada. O defeito de lógica de negócio permaneceu invisível tanto para os testes como para a análise estática, só a revisão humana com conhecimento do domínio poderia tê-lo identificado.

6.7 PR11 - Falso Positivo e Override

Este cenário documenta um caso de falso positivo com *override* deliberado. Foi desenvolvida uma nova funcionalidade (`add_detailed_tech_product`) para o registo de equipamentos tecnológicos, com testes que cobriam todos os requisitos funcionais.

- **Gate manual:** todos os testes passaram (7/7) com cobertura de 75%. Nenhum defeito funcional foi identificado.
- **Gate IA:** o SonarCloud reprovou com Classificação de Manutenibilidade D, devido ao elevado número de parâmetros da função e à presença de código comentado e *TODO comments*.
- **Decisão:** *override* - o programador efectuou o *merge* apesar da reprovação do *gate* com IA.

Do ponto de vista funcional, este é um caso de falso positivo: o código cumpria os requisitos de negócio e os testes passaram na totalidade. Os problemas de manutenibilidade identificados pelo SonarCloud eram tecnicamente válidos, mas o seu impacto operacional foi considerado aceitável como dívida técnica consciente. Este cenário ilustra que o julgamento humano é indispensável quando o *gate* com IA reprova por razões estruturais que não comprometem a funcionalidade.

6.8 PR15 - Falso Negativo

Este é o cenário mais crítico do estudo. Foi introduzido um defeito lógico no método `apply_discount`: em vez de subtrair o desconto ao preço, a função somava-o. Para simular um falso negativo realista, foi criado um teste que validava explicitamente o resultado incorrecto.

Listing 2 Defeito no método `apply_discount`

```
def apply_discount(self, name, discount_percent):
    if name not in self.products:
        raise KeyError(self.ERR_NOT_FOUND)
    price = self.products[name]["price"]
    # bug: devia ser price - (price * discount_percent / 100)
    return price + (price * discount_percent / 100)
```

Listing 3 Teste que valida o comportamento incorreto

```
def test_apply_discount():
    inv = Inventory()
    inv.add_product("Headset Bluetooth", 10, 100)
    result = inv.apply_discount("Headset Bluetooth", 10)
    # resultado correcto seria 90.0 (desconto de 10%)
    # o teste valida 110.0 — comportamento defeituoso
    assert result == 110.0
```

- **Gate manual:** todos os testes passaram, incluindo `test_apply_discount`. Cobertura de 76,32%.
- **Gate IA:** o SonarCloud aprovou o *Pull Request* sem identificar problemas. O código era sintaticamente limpo e a cobertura do código novo atingiu 80%.
- **Decisão:** ambos os *gates* aprovaram. O defeito só foi identificado através de revisão manual do código.

Este resultado representa um falso negativo puro: um erro financeiro real passou por todos os mecanismos automáticos sem ser detectado. A causa principal foi o teste validar o resultado errado, levando tanto o *pytest* como o SonarCloud a concluir que a implementação estava correcta.

6.9 Tabela de Resultados

A Tabela 3 resume os resultados de todos os *Pull Requests* analisados.

6.10 Análise Gráfica

A Figura 2 apresenta os resultados de cada *gate* por *Pull Request*. A Figura 3 mostra a matriz de confusão do *gate* com IA, usando o *gate* manual como referência. A Figura 4 ilustra a distribuição de concordâncias e *overrides*. A Figura 5 apresenta a cobertura de código por PR em relação ao limiar de 80% do SonarCloud.

Table 3 Resultados consolidados de todos os *Pull Requests*

PR	Cenário	Manual	IA	Override	Cobertura	Bugs
5	Baseline	PASS	PASS	Não	74,19%	0
6	Bug Funcional	FAIL	FAIL	Não	74,00%	—
7	Validação em Falta	PASS	PASS	Não	75,86%	0
8	Duplicação de Código	PASS	FAIL	Não	53,19%	0
9	Cobertura Baixa	PASS	FAIL	Não	60,00%	0
10	Defeito Subtil	PASS	FAIL	Não	76,32%	1
11	Falso Positivo	PASS	FAIL	Sim	75,00%	0
15	Falso Negativo	PASS	PASS	Não	76,32%	1

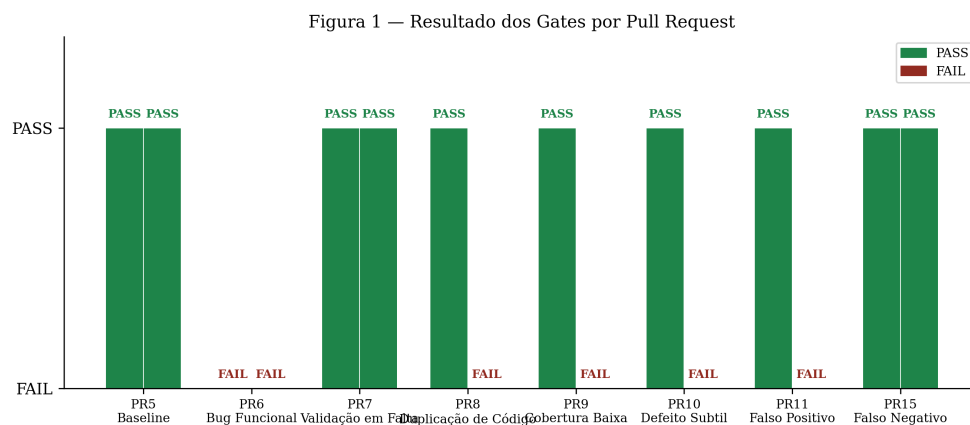


Fig. 2 Resultado dos *gates* manual e com IA por *Pull Request*. A verde: aprovação. A vermelho: reprovação.

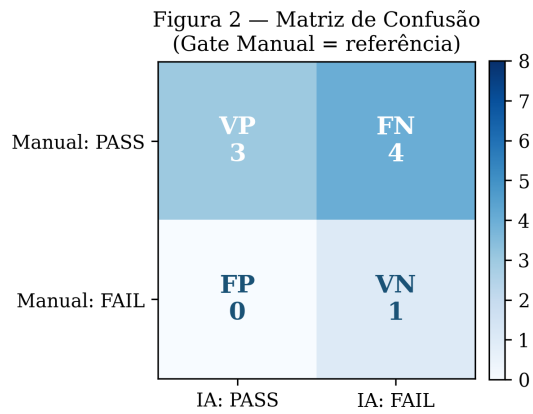


Fig. 3 Matriz de confusão do *gate* com IA. VP: verdadeiro positivo; VN: verdadeiro negativo; FP: falso positivo; FN: falso negativo. O *gate* manual é usado como referência.

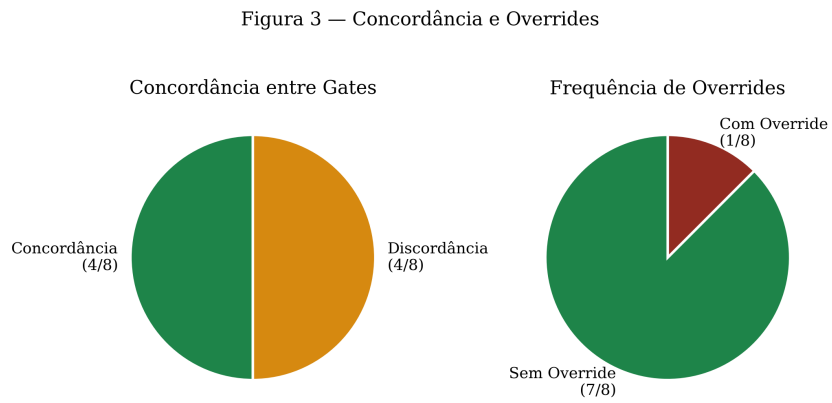


Fig. 4 Concordância entre os *gates* (esquerda) e frequência de *overrides* (direita) ao longo dos oito cenários.

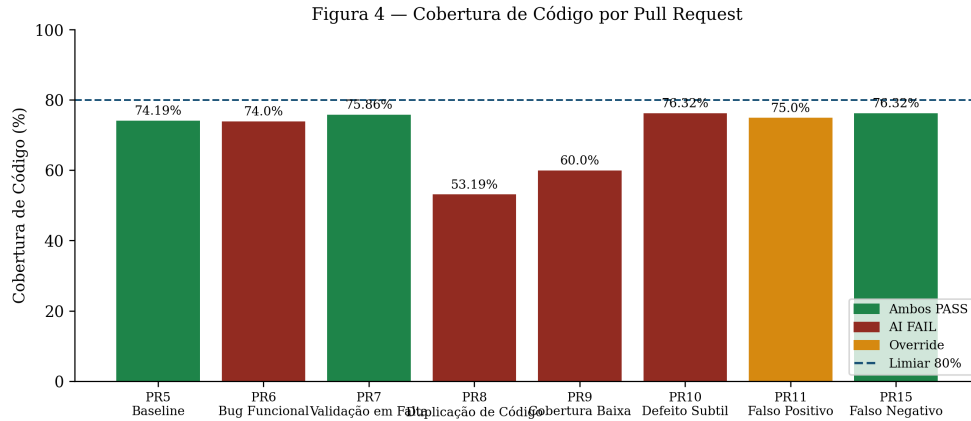


Fig. 5 Cobertura de código por *Pull Request*. A linha a tracejado representa o limiar de 80% configurado no SonarCloud.

7 Análise dos Resultados

A interpretação dos resultados visa esclarecer o comportamento dos dois mecanismos de validação em estudo (*Gate* Manual e o *Gate* com IA) durante a execução dos oito cenários experimentais previstos no planeamento metodológico. Em cada *Pull Request* foi feita uma modificação intencional no código, tornando possível verificar como cada *gate* responderia a defeitos funcionais, problemas de qualidade de código, problemas técnicos ou lógicos de negócio.

Nesta secção, os resultados são apresentados de maneira estruturada, inicialmente discutindo-se o desempenho do *gate* em relação à primeira RQ, que consiste na avaliação da influência da utilização de *quality gates* com IA sobre os resultados dos *pipelines*, e posteriormente analisando-se a segunda RQ, referente ao número de *overrides* e ao acerto das decisões tomadas, envolvendo falsos positivos e falsos negativos.

7.1 RQ1 - Impacto dos Gates com IA nos Resultados do Pipeline

Ao longo dos oito cenários, o *gate* manual aprovou 7 dos 8 *Pull Requests* (87,5%), enquanto o *gate* com IA aprovou apenas 3 (37,5%). Os dois *gates* concordaram em 4 dos 8 casos (50%). Esta diferença não significa que o *gate* com IA seja mais rigoroso de forma geral - significa que os dois *gates* avaliam dimensões diferentes do código.

O *gate* manual detecta bem os defeitos funcionais que os testes conseguem capturar. O *gate* com IA detecta bem os problemas estruturais: duplicação, cobertura insuficiente de código novo e manutenibilidade degradada. Os cenários de PR8 e PR9 são exemplos claros desta complementaridade: o *pytest* não identificou nenhum problema (porque os testes existentes continuavam a passar), mas o SonarCloud bloqueou a integração por razões estruturais válidas.

7.2 RQ2 - Frequência de Overrides e Decisões Falsas

Foi registado um *override* (PR11), correspondente a 12,5% dos *Pull Requests*. Este *override* foi motivado por um falso positivo: o SonarCloud reprovou código funcionalmente correcto com base em critérios de manutenibilidade. O programador, com conhecimento do contexto do projecto, considerou os problemas estruturais como dívida técnica aceitável e efectuou o *merge*.

O caso mais relevante para a RQ2 é o falso negativo do PR15. Tanto o *gate* manual como o *gate* com IA aprovaram um *Pull Request* com um defeito financeiro real. A análise da matriz de confusão (Figura 3) revela que o *gate* com IA obteve uma precisão de 1,00, um *recall* de 0,43 e um F1 de 0,60, o que reflecte exactamente este padrão: quando o SonarCloud reprova, reprova correctamente; mas deixa passar situações que o *gate* manual teria bloqueado.

7.3 Falso Negativo: Limitação Crítica

O PR15 expõe uma limitação fundamental das ferramentas de análise estática: a *cegueira semântica*. O SonarCloud avaliou a sintaxe, a estrutura e a cobertura do código novo, e tudo estava conforme as regras. Mas não tem forma de saber que `price + (price * discount / 100)` é semanticamente errado num contexto de desconto comercial.

Esta limitação não é exclusiva do SonarCloud, é inerente à análise estática enquanto técnica. A ferramenta não compreende o domínio de negócio, e apenas verifica se o código segue as regras estruturais que foram configuradas. O julgamento sobre a correcção da lógica de negócio continua a depender de quem escreve os testes e de quem revê o código.

7.4 Compromissos Identificados

Os compromissos principais identificados durante a realização destes oito cenários experimentais são apresentados na Tabela 4. Para cada um desses compromissos, há um aspecto do problema onde ambos os *gates* possuem vantagens, mas também limitações. Estes resultados ajudam a compreender porque é que, em certas circunstâncias, os *gates* são concordantes, enquanto em outras são discordantes e necessitam de intervenção humana, como ocorreu com o *override* de PR11.

7.5 Ameaças à Validade

Como recomendado na literatura de investigação empírica em engenharia de software, é essencial identificar potenciais ameaças à validade, de forma a clarificar em que medida os resultados podem ser generalizados ou influenciados por fatores externos. Nesta subsecção analisam-se três dimensões fundamentais de validade (interna, externa e de construção) e descreve-se como cada uma pode ter impactado as conclusões deste trabalho.

- **Validade interna:** os cenários foram deliberadamente concebidos para produzir resultados específicos. Um estudo com desenvolvimento orgânico de funcionalidades produziria padrões de discordância diferentes.

Table 4 Compromissos identificados entre os dois tipos de *gate*

Compromisso	Observação
Velocidade vs qualidade	O <i>gate</i> com IA acrescenta tempo ao pipeline mas detecta problemas que os testes tradicionais não capturam.
Confiança vs automação	O <i>override</i> do PR11 mostra que os programadores ignoram o <i>gate</i> com IA quando consideram que este está errado.
Cobertura vs correcção	Uma cobertura de 76% e zero problemas no SonarCloud não impediram o falso negativo do PR15.
Estrutura vs semântica	O SonarCloud é eficaz na detecção estrutural mas cego à lógica de negócio.

- **Validade externa:** o estudo usa uma aplicação Python sintética de pequena dimensão com um único programador. Os resultados podem não generalizar para projectos de maior escala ou equipas com múltiplos elementos.
- **Validade de construção:** o *gate* manual é usado como referência, mas reflecte apenas o que os testes cobrem. Como o PR15 demonstra, um teste pode passar enquanto valida comportamento incorrecto.

8 Conclusões

Este trabalho apresentou um estudo comparativo entre *gates* de qualidade manuais e assistidos por IA, desenvolvido sobre um pipeline real com GitHub Actions e SonarCloud, ao longo de oito cenários estruturados de *Pull Request*.

Relativamente à RQ1, os resultados mostram que o *gate* com IA e o *gate* manual concordaram em 50% dos casos, com a discordância a concentrar-se em cenários de qualidade estrutural, tais como duplicação de código, cobertura insuficiente e manutenibilidade degradada. Isto confirma que os dois *gates* são complementares: o manual avalia a correcção funcional, o de IA avalia a qualidade estrutural.

Relativamente à RQ2, foi registado um *override* (12,5%) motivado por um falso positivo, onde o SonarCloud reprovou código funcionalmente correcto por razões de manutenibilidade. O caso mais crítico foi o falso negativo do PR15, onde ambos os *gates* aprovaram código com um defeito financeiro real, um defeito que só seria detectável através de revisão humana com conhecimento do domínio de negócio.

A principal conclusão deste trabalho é que os *gates* de qualidade com IA são uma ferramenta valiosa mas com limitações claras. São eficazes na detecção de problemas estruturais, rápidos e consistentes. Mas são semanticamente cegos, pois não compreendem o que o código deve fazer, apenas verificam se está estruturado de acordo com as regras configuradas. A supervisão humana continua a ser indispensável, especialmente para validar a correcção da lógica de negócio.

Em trabalho futuro, seria interessante estender este estudo a projectos reais de maior dimensão, explorar o efeito da calibração dos limiares do SonarCloud na frequência de *overrides*, e investigar de que forma as equipas de desenvolvimento desenvolvem (ou perdem) confiança nos *gates* automatizados ao longo do tempo.

References

- [1] Duvall, P.M., Matyas, S., Glover, A.: Continuous Integration: Improving Software Quality and Reducing Risk. Addison-Wesley, Upper Saddle River, NJ (2007). <https://dokumen.pub/continuous-integration-improving-software-quality-and-reducing-risk-8-printnbsped-9780321336385-032133638.html>
- [2] Humble, J., Farley, D.: Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley, Upper Saddle River, NJ (2010). <https://github.com/aaquresh/CITraining/blob/master/Ebooks/Continuous%20Delivery%20-%20Reliable%20Software%20Releases%20Through%20Build,%20Test%20And%20Deployment%20Automation.pdf>
- [3] SonarSource: SonarCloud Documentation. <https://docs.sonarcloud.io>. Accessed: 2025-05-01 (2024)
- [4] Zampetti, F., Scalabrino, S., Oliveto, R., Canfora, G., Di Penta, M.: How Open Source Projects Use Static Code Analysis Tools in Continuous Integration Pipelines. IEEE (2017). https://www.researchgate.net/publication/318127552_How_Open_Source_Projects_Use_Static_Code_Analysis_Tools_in_Continuous_Integration_Pipelines
- [5] Hilton, M., Tunnell, T., Huang, K., Marinov, D., Dig, D.: Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects. ACM (2016). <https://mir.cs.illinois.edu/marinov/publications/HiltonETAL16ContinuousIntegration.pdf>
- [6] Vassallo, C., Schermann, G., Zampetti, F., Romano, D., Leitner, P., Zaidman, A., Di Penta, M., Panichella, S.: Every Build You Break: Developer-Oriented Assistance for Build Failure Resolution. Springer (2020). https://www.researchgate.net/publication/336390136_Every_Build_You_Break_Developer-Oriented_Assistance_for_Build_Failure_Resolution